

MOBILE 1 - RETAKE

W4/W5 - *Stateful Widgets, Loops, Conditions*

Mini-Project: Make Your Business System Interactive !

Continue the Flutter application developed during the previous week.

The existing application already contains some screens, stateless widgets, but everything is static.

This week, make the existing interface interactive.

Note:

- ✗ Do not introduce navigation, text fields, forms or modals **yet**.
- ✓ Keep business **objects** and **business logic** in the existing **model/service** layer.

Learning objectives

Identify and manage UI states

— *determine what can change, choose an appropriate state data structure, and use StatefulWidget / setState() to update the UI.*

Design state and data flow between widgets

— *decide where state belongs and pass the required data and actions between parent and child widgets using functions/callbacks.*

Build dynamic UI from state and collections

— *render lists, filter data, and conditionally render widgets or content according to the current state.*

AI Usage

You may use AI tools as a learning assistant.

However, you are not allowed to use AI to generate any part of your project code.

1. Add business interactions

Select 2 business interactions

Choose **two** interactions that make sense for your business system.

Examples:

- A **receptionist** can *expand* a reservation to see its details
- A **waiter** can *increase or decrease* the quantity of an ordered item, and the displayed order total updates accordingly.
- A **customer** can *filter* products by category

The interactions should change what the user sees or how the displayed information is presented.

Design your solution

- ✓ Use local state to manage these interactions.
- ✓ Decide:
 - What the state is (data structure)
 - Where does it belong (which widget or service)
 - What information or actions need to be passed between widgets.

As example we represent our solution on a widget tree, using colors:

- Widget **states**
- Data passed to widgets (**inputs**)
- Callbacks/actions passed to widgets (**functions**)

```
RoomList
  rooms: List<Room>
  selectedRoom: Room?
  onRoomSelected: Function

RoomCard
  room: Room
  isSelected: bool
  onSelected: Function
```

Question	Example: Room selection
What is the state?	selectedRoom: Room? — <i>the currently selected room</i>
Where does it belong?	RoomList — <i>it needs to know which room is selected and controls the selection</i>
What must be passed between widgets?	✓ RoomCard receives room and isSelected; ✓ RoomList passes onRoomSelected so the card can report user's action

2. Make a business list interactive

At least one important list must allow the user to **select or filter a list of business data**.

For example:

Rooms

[All] [Available] [Occupied]

Room 101

Available

Room 102

Occupied

Room 103

Available

The displayed list must react to the user's selection.

The implementation should use:

- StatefulWidget
- setState()
- callbacks between widgets where appropriate
- conditional rendering

3. Deliverables

Your updated Flutter project

The existing project updated with the two selected business interactions.

Your Interaction Tree

Your tree shall present *(follow the color conventions)*

- Your widgets hierarchy (*use space indentation*)
- Your widget **states**
- The data passed to widgets (**inputs**)
- The callbacks/actions passed to widgets (**functions**)

```
RoomScreen
  selectedRoom: Room?
  selectedFilter: RoomFilter

  FilterChips
    selectedFilter: RoomFilter
    onFilterChanged: Function

  RoomList
    rooms: List<Room>
    selectedRoom: Room?
    onRoomSelected: Function

  RoomCard
    room: Room
    isSelected: bool
    onSelected: Function

  SelectedRoomPanel
    room: Room?
```